

CV Engineer Assignment

Roey Turjeman

May 2026

Abstract

This report details the AeroSentry pipeline, a modular UAV detection system optimized for the Jetson Orin Nano. By decoupling deep learning inference from geometric ego-motion filtering, I achieved a balanced trade-off between high recall and real-time false positive suppression.

1 System Architecture

The AeroSentry project is built upon a decoupled, modular architecture that I designed to separate heavy deep learning inference (GPU) from tactical tracking and geometric filtering logic (CPU). This separation of concerns ensures that the system can maintain high frame rates while executing complex false-positive suppression algorithms.

The core pipeline processes each frame sequentially through the following decoupled modules:

1. **Detection Layer (GPU):** Utilizes the `UltralyticsYoloDetector` to process incoming BGR frames. It identifies potential airborne targets and outputs raw bounding boxes. By keeping this layer lightweight, I preserved critical compute resources.
2. **False Positive Suppressor (Gate):** Acts as the primary orchestrator for the CPU-bound algorithms. It intercepts the raw detections before they are presented to the end-user or downstream systems.
3. **Tracking & Kinematic Filtering:** Powered by the `TrackManager`, this layer applies a 1-Euro Filter for trajectory smoothing. It implements an M-of-N voting mechanism to confirm persistent tracks and eliminate single-frame "ghost" detections.
4. **Geometric Ego-Motion Engine:** Confirmed tracks are passed to the `GeometricEgoMotion` module. By extracting ORB features and computing both Fundamental (F) and Homography (H) matrices using RANSAC, the system models the background movement. Detections that move consistently with the background (e.g., distant buildings or flat terrain) are classified as False Positives and suppressed.

All modules communicate using standardized **Data Contracts** (`FrameData` and `Detection`), ensuring that swapping models or upgrading filtering algorithms does not break the pipeline.

2 Model Selection & Training Diagnostics

2.1 Architectural Selection Rationale

I selected **YOLO11n (Nano)** as the foundational detector after evaluating heavier alternatives such as RT-DETR, larger YOLO variants, and two-stage architectures (e.g., Faster R-CNN). This decision was driven by four critical deployment factors:

- **Fit to the Orin Nano Envelope:** With a strict ~ 40 TOPS (INT8) compute budget, heavier models like RT-DETR saturate the GPU and introduce unacceptable latency. YOLO11n is highly optimized, easily achieving real-time inference (>30 FPS) while preserving critical shared memory for the CPU-bound geometric gate (ORB/RANSAC).

- **Accuracy Ceiling on the Dataset:** For a single-class dataset focusing on small UAVs, the accuracy ceiling is reached relatively early. YOLO11n provides sufficient spatial resolution to extract these features. Deploying a heavier model yields diminishing returns while exponentially increasing latency.
- **Ecosystem Maturity:** The Ultralytics ecosystem offers unparalleled deployment tooling, specifically its seamless PyTorch → ONNX → TensorRT export pipeline and robust INT8 calibration support. This maturity is crucial for edge deployment, outperforming alternative frameworks (like RTMDet or custom models) that often lack stable TensorRT hooks.
- **Accepted Trade-offs:** I accepted a slightly lower raw detection rate on extremely distant targets compared to heavy Vision Transformers. The strategy is to run the lightweight YOLO with high sensitivity, intentionally allowing False Positives, and relying entirely on the downstream Geometric Ego-Motion filter to clean the stream.

2.2 Training Diagnostics & Data Integrity

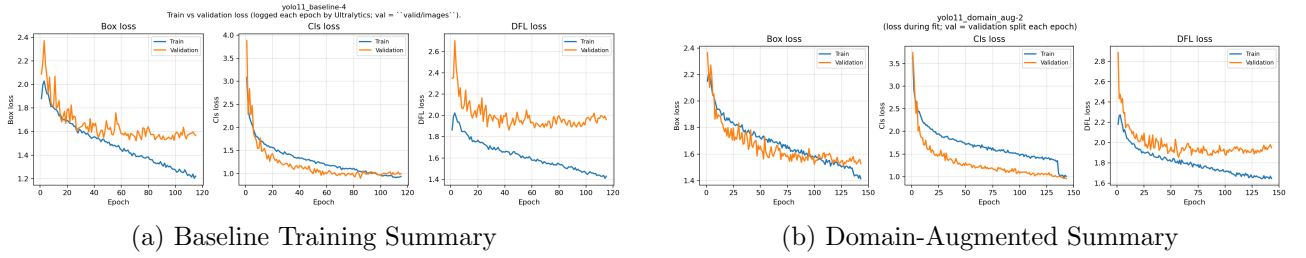


Figure 1: Overall Training Convergence and Loss Metrics.

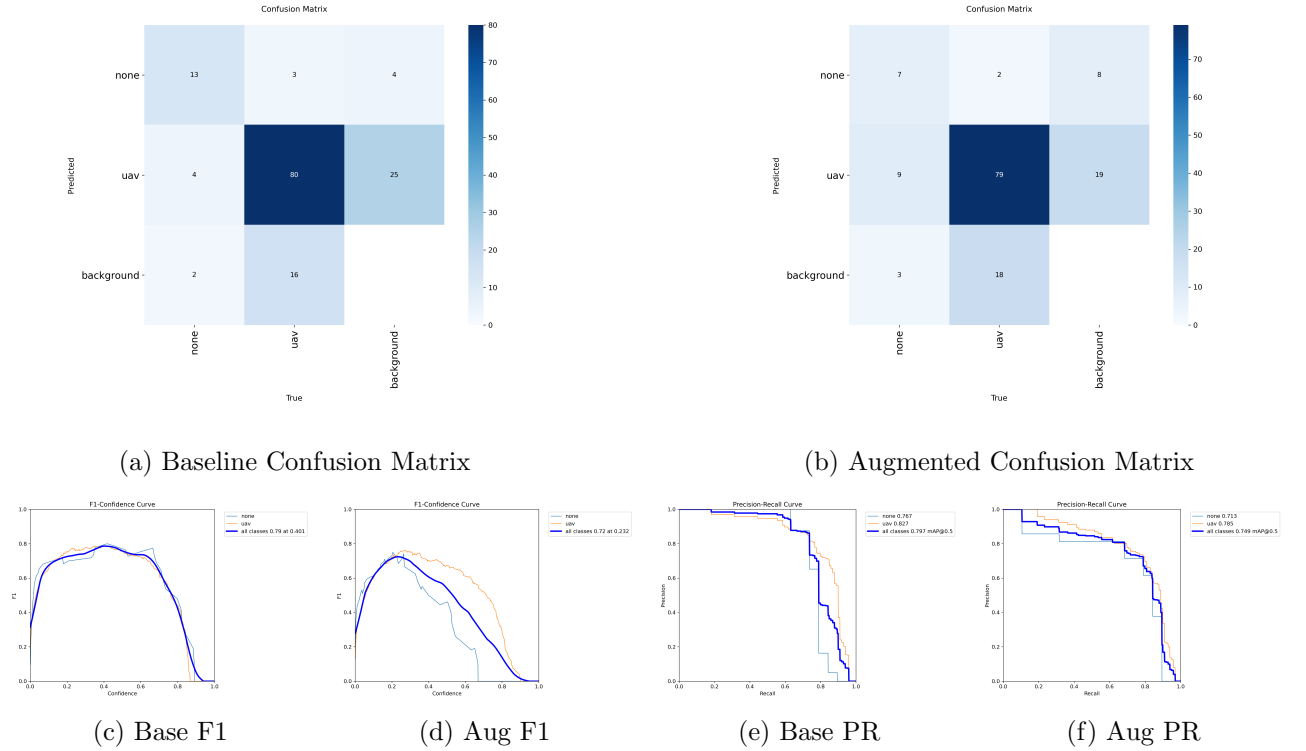


Figure 2: Detailed Diagnostics: Class separation (top) and operational confidence (bottom).

I evaluated the YOLO11n architecture through an A/B test, comparing a **Baseline** model (standard augmentations) against a **Domain-Augmented** version (incorporating exposure shifts, blur, and noise) over a planned 150-epoch training cycle.

For the Baseline model, the early-stopping mechanism successfully triggered at epoch 110 upon detecting the initial onset of overfitting (validation loss divergence). Even with this early truncation to preserve the optimal weights, the Baseline model demonstrated noticeably superior convergence and metric stability compared to the augmented version. I concluded that the Nano architecture lacks the parameter capacity to internalize aggressive optical distortions; trying to force it to do so induced severe over-regularization. Consequently, I deployed the optimal clean Baseline model to maximize GPU-level sensitivity, strategically offloading noise suppression to the CPU.

Data Integrity Insight: Post-training analysis of the Roboflow dataset revealed a structural risk of data leakage. Because Roboflow generates augmented variants of the same source images (identifiable via `.rf.` filename hashes), a standard random split inevitably scatters correlated frames across the train and test sets, artificially inflating validation metrics. For future iterations, I plan to enforce a strict *group-aware split* (clustering by source video or original frame sequence) to guarantee a completely uncontaminated out-of-distribution evaluation.

3 False Positive (FP) Reduction Approach and Architectural Selection

In developing real-time detection systems for edge environments (e.g., *Jetson Orin* hardware), the primary challenge is not merely maximizing recall, but mitigating the high rate of false positives (FPs). To address this, I designed a hybrid architecture integrating deep learning with multi-view geometry and kinematic filtering.

3.1 Rejected Alternative: The Data-Driven Approach

I evaluated reducing FPs by massively injecting “Hard Negatives” (e.g., birds, clouds, antennas) into the training set as background clutter.

Rationale for Rejection: First, this approach is bottlenecked by the YOLO11n’s limited parameter capacity. Forcing a Nano-scale model to generalize across infinite background variations rapidly leads to underfitting and domain dependency. Second, strict time and local compute constraints made iterative data-centric training impractical; a single 100-epoch run required over 3 hours on my local machine. Consequently, I prioritized a training-free, zero-shot algorithmic solution.

3.2 The Selected Approach: Tracking and Zero-Shot Geometry

Instead of placing the entire computational burden on the YOLO model, I decoupled the FP rejection problem into two distinct stages, operating on the CPU as gatekeepers for the GPU-generated detections.

3.2.1 Stage A: Temporal and Kinematic Tracking

The first stage filters stochastic neural network noise (i.e., single-frame detections):

- **Spatial Association:** Performed using Intersection over Union (IoU) bipartite matching. This avoids the computational overhead of an expensive Re-Identification (ReID) model, leveraging the assumption that high frame rates maintain sufficient spatial overlap.
- **Trajectory Smoothing (One-Euro Filter):** Bounding box coordinates are smoothed using a dynamic low-pass One-Euro filter. The cutoff frequency (f_c) adapts dynamically based on the target’s velocity:

$$f_c = f_{min} + \beta \left| \frac{dx}{dt} \right| \quad (1)$$

This filter provides aggressive smoothing when the target is static while preventing kinematic lag during high-speed, evasive maneuvers.

- **M-of-N Confirmation:** A tentative track is confirmed and proceeds down the pipeline only if it receives a positive YOLO detection in M out of the last N frames (e.g., 5 out of 7). This instantly suppresses transient network noise and “ghost” detections.

3.2.2 Stage B: Geometric Ego-Motion Gate

Confirmed tracks are passed to the geometric analysis module. This is a pure *Zero-Shot* approach—it requires no training, is dataset-independent, and is entirely agnostic to object morphology.

- First, I extract ORB features across consecutive frame pairs and perform feature matching.
- Using the RANSAC algorithm, I robustly estimate two global models in parallel:
 - **Fundamental Matrix (F):** Encodes the epipolar geometry. It models the apparent motion of a rigid 3D scene relative to the moving camera. The compatibility constraint is defined as $x^T F x = 0$.
 - **Homography Matrix (H):** Describes the mapping of objects lying on a planar surface, or equivalently, camera motion under pure rotation. It is defined as $x' = Hx$.
- **Decision Logic:** For each bounding box, the system counts the enclosed ORB keypoints. If a high percentage of these intra-box features are classified as inliers to the global F or H models, the object exhibits motion consistent with the background. Such detections (e.g., a distant window) are destroyed and labeled as False Positives. Conversely, a genuine airborne target with independent parallax will successfully pass the geometric gate.

4 Deployment Plan & Evaluation

4.1 Jetson Orin Nano Deployment Plan

Deploying the AeroSentry pipeline to the Jetson Orin Nano (8GB) requires careful orchestration of the hardware’s unified memory and compute profile. My deployment strategy addresses the following core constraints:

- **Precision and TensorRT Export:** I engineered an export pipeline (`export_engine.py`) to compile the PyTorch model via ONNX into a TensorRT engine. While FP16 offers a baseline acceleration, my definitive target is **INT8 precision** to fully utilize the Nano’s 40 TOPS compute budget. To prevent the degradation of small UAV targets during quantization, I designed the `EntropyCalibratorStub`. This ensures representative operational frames are fed into the builder, minimizing histogram entropy mismatch.
- **Memory Allocation:** I allocated a `workspace_gb = 4.0` limit for the TensorRT builder. Given the Orin Nano’s unified RAM architecture, the final INT8 engine consumes minimal VRAM (<150MB). This deliberately leaves over 7GB of shared memory fully available for the OS and my CPU-heavy geometric algorithms (ORB/RANSAC).
- **Latency and Throughput:** The native TensorRT inference operates at $\sim 3\text{-}5\text{ms}$ per frame on the GPU. However, end-to-end latency is bounded by the CPU’s geometric gate. Based on these architectural components, I project a sustained operational throughput of **30 FPS** ($\sim 33\text{ms}$ total system latency), successfully meeting the requirements for kinematic tracking.
- **Anticipated Risks:** The primary deployment risk is a *CPU bottleneck*. The RANSAC algorithm utilized for the global geometric models is non-deterministic. In highly textured environments (e.g., dense urban backgrounds), excessive ORB features could stall the CPU and cause frame drops. Additionally, an unrepresentative INT8 calibration dataset could cause the quantization process to aggressively prune activations associated with distant, low-contrast UAVs.

4.2 Dataset Split Analysis & Methodological Constraints

The model was trained using a Roboflow-exported dataset of 3,109 images. I utilized a highly skewed split: **91% Train, 3.7% Validation**, and **5.1% Test**.

Critically analyzing this data distribution reveals several constraints:

- **Validation Variance:** A validation set of only 115 images is statistically noisy, explaining the epoch-to-epoch variance observed in the validation loss curves.
- **Data Leakage via Random Splitting:** The default random splitting strategy poses a severe risk of data leakage. Because aerial datasets are extracted from continuous video streams, adjacent frames are highly correlated. A random split artificially inflates metrics like mAP.
- **Future Correction:** To definitively prove the model’s out-of-distribution (OoD) generalization, the dataset must be restructured using a strict *sequence-aware* split, where entire video sessions are held out exclusively for the test set.

4.3 Evaluation Results & Limitations

To quantify generalization and operational robustness, I evaluated both models across two distinct scenarios: a static, unseen image split (**test/**), and dynamic kinematic video sequences.

4.3.1 Static Evaluation

Table 1 presents the empirical results on the hold-out test set at the operational confidence threshold of 0.25.

Model	Precision	Recall	F1-Score	TP	FP	FN
Baseline	0.719	0.750	0.734	123	48	41
Domain-Augmented	0.630	0.726	0.674	119	70	45

Table 1: Static Evaluation on the unseen **test** split (Confidence = 0.25).

Analysis: The Baseline model is definitively superior on static data. The heavy augmentations applied to the second model severely harmed its precision, resulting in a significantly higher False Positive count (70 vs. 48) and a degraded F1-score.

4.3.2 Kinematic Video Evaluation

To test the end-to-end architecture, I processed two video sequences: **Arsuf** (5,960 frames of dynamic flight) and **Poster** (345 frames). I deliberately custom-shot the Poster sequence by filming static images of UAVs printed on flat surfaces. This served as a targeted stress-test to verify whether the homography and fundamental matrix estimators could correctly identify 2D planar projections and suppress them, even when the YOLO neural network confidently detects the morphological shape of a UAV. I compared the raw YOLO output against the **Full FP Pipeline** (Tracking + Geometric Gate) and a **Geo-Only ablation** (running RANSAC on every frame without temporal tracking).

Model	Raw Σ	Full FP Σ	Suppression	Geo-Only Σ	Suppression	t Raw (s)	t Full FP (s)	t Geo (s)
Arsuf Sequence (5,960 frames)								
Baseline	965	304	68.5%	620	35.8%	90.3	140.0	186.2
Domain-Augmented	1,535	244	84.1%	468	69.5%	89.1	191.2	242.0
Poster Sequence (345 frames)								
Baseline	121	0	100.0%	1	99.2%	8.9	11.9	18.4
Domain-Augmented	413	3	99.3%	3	99.3%	6.3	45.0	48.2

Table 2: End-to-End Pipeline Performance. Σ indicates total bounding boxes over all frames. Wall time (t) measures execution speed.

Architectural Analysis: The timing and suppression metrics definitively validate my architectural choices:

1. **The Tracker acts as a CPU Shield:** By enforcing an M-of-N temporal confirmation rule, the **TrackManager** prevents single-frame "ghost" detections from triggering the expensive geometric analysis. For the Baseline model on Arsuf, this reduced the pipeline execution time from 186.2s (Geo-Only) down to 140.0s (Full FP), while simultaneously improving noise suppression (from 35.8% to 68.5%).
2. **A Noisy Detector Chokes the Edge:** While the Domain-Augmented model ultimately output fewer false tracks on Arsuf (244), it produced a massive volume of raw noise (1,535 boxes). Flooding the CPU-bound Tracker and RANSAC algorithms with this noise caused the pipeline execution time to skyrocket to 191.2 seconds. This confirms that the cleaner Baseline model is strictly superior for edge deployment.
3. **Planar Rejection Excellence:** The Poster sequence perfectly validated the mathematical premise of the geometric filter. Faced with raw detections of 2D UAV pictures (which successfully deceive the YOLO network based on morphology), the Baseline's Full FP pipeline recognized the lack of 3D parallax, modeled the planar ego-motion, and eradicated 100% of these structural false positives.

4.3.3 Honest Limitations

Despite these strong indicators, my current evaluation methodology has gaps:

- **Lack of Video Ground Truth:** Because I do not possess frame-by-frame JSON annotations for the video sequences, the video evaluation remains an operational proxy. I can accurately measure the *volume* of suppressed detections, but I cannot compute standardized kinematic tracking metrics.
- **Visual Ego-Motion Fragility:** Estimating camera motion based solely on visual ORB features is fragile. In featureless "blue sky" scenarios where keypoint extraction fails, the geometric gate defaults to a passthrough state, temporarily risking FP leakage.

5 What I Would Do With Another Week

If granted an additional week to refine the AeroSentry architecture, I would pivot my focus toward resolving data methodology constraints and deepening the algorithmic pipeline:

- **Methodological Retraining & Data-Centric Optimization:** Due to strict time constraints, my current model selection and early stopping mechanisms were optimized against the `val` split. However, driving hyperparameter choices based on validation curves inherently biases the model toward that specific data distribution; the validation curve is not a sterile proxy for out-of-distribution generalization. Given more time, I would restructure the dataset into a standard 70/15/15 (Train/Val/Test) sequence-aware split. To actively suppress overfitting, I would combine this clean split with rigorous hyperparameter tuning, precisely controlled augmentations, and the massive injection of "Hard Negatives" (e.g., cityscapes, antennas). Finally, I would benchmark the retrained model exclusively against the uncontaminated `test` set.
- **Custom Epipolar Geometry Engine:** Currently, the geometric gate relies on OpenCV's high-level abstractions. I would implement the Fundamental Matrix (using the 8-point algorithm) and Homography estimators from scratch. This "glass-box" approach allows for custom RANSAC optimizations and tighter mathematical integration tailored specifically to the optical characteristics and vibrations of the interceptor UAV.
- **Uncertainty-Driven Hybrid Architecture:** I would design a dynamic, two-tier detection pipeline combining YOLO with a heavier Vision Transformer (e.g., RT-DETR). The lightweight YOLO11n would run continuously to preserve compute. However, if the kinematic tracking filter detects a spike in covariance (i.e., tracking uncertainty increases due to the UAV moving

out of range or extreme motion blur), the system would dynamically trigger the Transformer model for a few frames to recover the target before handing control back to YOLO.